

# Lab exercises

Part 11 —  The LLM Model Landscape · 11-llm-models/lab/EXERCISES.md

## Lab — The LLM Model Landscape

Work top to bottom. Each exercise has a **stated deliverable**; if you can't produce it, you haven't finished. No API keys are required — everything runs offline against the code in `../code/` or against your own reasoning and research.

### 1. Tier-map a real product line (30 min)

Pick one provider from notes/02's snapshot table [11-llm-models/notes/02-selecting-and-migrating-models.md] that you have NOT used before. Go to its official docs/pricing page.

- **1a.** Identify its frontier, mid, and small tiers by name, and one reasoning-mode or reasoning-variant offering if it has one.
- **1b.** Write down the current context window and per-million-token input/output price for each tier.
- **1c.** State the date you checked. This column is the entire point of this exercise: a model landscape note without a "checked on" date is worthless in three months.

**Deliverable:** a 4-row table (3 tiers + reasoning variant), with a checked-on date.

### 2. Score a real (or realistic) task (1h)

Open `code/model_selection_scorer.py`.

- **2a.** Define a `TaskProfile` for a use case you actually care about — a real one from work, or one of: "internal Slack bot that answers HR policy questions," "batch-classify 2M support tickets by category overnight," or "real-time code-completion in an IDE."
- **2b.** Fill in 3-4 `CandidateModel` entries using numbers you found in exercise 1 (or reasonable estimates, clearly marked as estimates) instead of the illustrative defaults.
- **2c.** Run the scorer. Which candidate wins? Change one weight by 0.1 — does the winner change? What does that tell you about how confident you should be in the recommendation?

**Deliverable:** your edited `TaskProfile` + `CandidateModel` list, the ranked output, and 2-3 sentences on whether the recommendation is robust or fragile to the weights you chose.

### 3. Break a leaderboard number (45 min)

Run `python code/benchmark_literacy_demo.py`.

- **3a.** In the contamination demo, explain in your own words why "heavily-contaminated" and "clean-model" have identical true skill but different public scores. What real-world action would catch this that a public leaderboard alone would not?
- **3b.** In the variance demo, find the smallest `n_problems` value where the 95% CIs stop overlapping. Compare that `n` to GPQA's actual question count (notes/01 section 3). What does that comparison tell you about trusting a GPQA score difference of 1-2 points between two models?
- **3c.** Change `model_b_skill` in the script to `0.85` (a large, real gap) and re-run just the `n=50` case. Does a big true gap resolve at small `n` where a small true gap couldn't? State why.

**Deliverable:** three short answers (3a-3c), each grounded in numbers the script actually printed.

---

#### 4. Proprietary vs. open-weights, defended (1h)

For each scenario, choose proprietary API or open-weights and defend it in  $\leq 5$  sentences using data residency, customization depth, cost-at-scale, and licensing — not vibes. At least one scenario should be a close call where you explicitly say what would flip your answer.

1. A healthcare startup summarizing de-identified clinical notes, EU-only infrastructure required by contract.
2. A consumer app with 50M requests/month of simple classification, no compliance constraints.
3. A defense contractor that legally cannot send any data to a third-party API, of any kind, ever.
4. A five-person startup prototyping a new feature, uncertain if it will ever reach meaningful volume.

**Checks:** - In #1 and #3, "proprietary API" should require an explicit private/on-prem justification if you pick it — otherwise you've ignored the stated constraint. - In #4 you should explain why premature self-hosting is very likely the wrong call, using the break-even logic from Module 04's cost calculator [04-

`llm/code/llm_cost_calculator.py`].

---

#### 5. The migration runbook, exercised (1.5h)

Take the winning candidate from exercise 2. Imagine its provider announces a deprecation date for the exact version you pinned, six months out.

- **5a.** Using notes/02 section 3's migration checklist [11-llm-models/notes/02-selecting-and-migrating-models.md], write the actual steps you'd take, in order, naming what you'd measure at each step.
- **5b.** Name the two failure modes from the "what breaks when you swap models" table (notes/02 section 3) most likely to bite YOUR specific use case from exercise 2, and why.
- **5c.** Write the one Slack message you'd send your team the day the deprecation notice arrives. It should reference a date, an owner, and where the eval baseline lives.

**Deliverable:** the ordered runbook, the two failure-mode picks with justification, and the Slack message.

---

## 6. Stretch: design the model gateway (1.5h)

Sketch (ASCII diagram or prose, no code required) an abstraction layer that sits between your application and model providers, per notes/02 section 3 [11-llm-models/notes/02-selecting-and-migrating-models.md] and Module 10 [10-ai-architecture].

- **6a.** What does the application ask the gateway for (a capability, not a model name)?
- **6b.** What does the gateway log on every call, and why does each field matter for catching a silent regression when a provider updates a "latest" alias?
- **6c.** Design the fallback behavior when the primary provider is down or rate-limited. What do you sacrifice (latency? consistency? cost?) to get resilience, and is that trade-off different for a real-time chat path vs. a batch pipeline?

**Deliverable:** the diagram/prose, the logged-field list, and the fallback design with its trade-off stated explicitly.